

# ■■ LOW-LEVEL DESIGN MASTERY

12-Week OOP → Patterns → Real Problems · Write Production Code · Interview Ready

by **SYNTAX ERROR** | [syntaxerrorr.com](https://syntaxerrorr.com) | [@code.abhii07](https://twitter.com/code.abhii07)

|       |        |         |          |
|-------|--------|---------|----------|
| 12    | 5      | 2–3 hrs | 50+      |
| Weeks | Phases | Daily   | Problems |

## ■ DAILY STUDY PLAN (2–3 hours)

|            |                   |                   |
|------------|-------------------|-------------------|
| 1 hr       | 1 hr              | 1 hr              |
| ■ Concepts | ■ Code & Practice | ■ Design Problems |

## ■ GOLDEN RULES — DESIGN FOR PRODUCTION

|   |                                                                         |
|---|-------------------------------------------------------------------------|
| ■ | Don't just study theory — write actual code for every concept you learn |
| ■ | Don't memorize patterns blindly — understand WHY each pattern exists    |
| ■ | Draw class diagrams first — visualize relationships before writing code |
| ■ | Think real-world — interview = design for production, not a toy project |

## ■ 5-PHASE ROADMAP

|                  |                  |                |                 |               |
|------------------|------------------|----------------|-----------------|---------------|
| P1               | P2               | P3             | P4              | P5            |
| OOP Fundamentals | SOLID Principles | UML & Thinking | Design Patterns | Real Problems |

## ■■ PHASE 1 — OOP Fundamentals (Most Important!)

### Phase 1 OOP FUNDAMENTALS · Week 1–2 · 2 Weeks

■ Classes & Objects — Blueprint vs Instance, memory, real-world modeling ■ Encapsulation — Data hiding, getters/setters, access modifiers (private/public) ■ Inheritance — Code reuse, parent-child relationships, method inheritance ■ Polymorphism — Method overloading (compile-time), method overriding (runtime) ■ Abstraction — Hiding complexity, abstract classes vs interfaces ■ Practice — Student class system, Bank Account system from scratch

## ■■ PHASE 2 — SOLID Design Principles

### Phase 2 SOLID PRINCIPLES · Week 3–4 · 2 Weeks

■ S — Single Responsibility — One class, one job, one reason to change ■ O — Open/Closed — Open for extension, closed for modification ■ L — Liskov Substitution — Child class can replace parent everywhere seamlessly ■ I — Interface Segregation — Specific interfaces over fat general-purpose ones ■ D — Dependency Inversion — Depend on abstractions, not concrete implementations ■ Bonus — DRY (Don't Repeat Yourself), KISS (Keep It Simple), YAGNI

## ■■ PHASE 3 — UML & Design Thinking

### Phase 3 UML & DESIGN THINKING · Week 5 · 1 Week

■ Class Diagrams — Visualizing classes, attributes, methods and relationships ■ Association — Objects work together without ownership ■ Aggregation — Has-a relationship (weak ownership) — Car has Wheels ■ Composition — Has-a relationship (strong ownership) — House has Rooms ■ Inheritance Diagrams — Parent-child relationships, interface implementation ■ Draw before coding — Always plan your design on paper/diagram first

## ■ ■ PHASE 4 — Design Patterns (Week 6–8)

### Phase 4 DESIGN PATTERNS · Week 6–8 · 3 Weeks

■ Creational — Singleton (one instance), Factory (object creation), Builder (complex objects) ■ Structural — Adapter (compatibility bridge), Decorator (add features), Proxy (control access) ■ Behavioral — Observer (notify multiple), Strategy (choose algorithm at runtime) ■ MVC Pattern — Model, View, Controller — separate concerns cleanly ■ Repository Pattern — Abstracting data access layer from business logic ■ When to use which pattern — context matters, don't force patterns everywhere

## ■ ■ PHASE 5 — Real LLD Problems (Week 9–12)

### Phase 5 PRACTICE DESIGNING REAL SYSTEMS · Week 9–12 · 4 Weeks

- |   |                                                                                                                    |
|---|--------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Parking Lot System</b> Multi-level, spot types (compact/large), payment system, available space search          |
| 2 | <b>Library Management</b> Books, members, checkout flow, reservation queue, fine calculation                       |
| 3 | <b>Elevator System</b> Multiple elevators, floor requests, scheduling algorithm, edge cases                        |
| 4 | <b>Movie Ticket Booking (BookMyShow)</b> Shows, seat selection, pricing tiers, booking flow, cancellation & refund |
| 5 | <b>Food Delivery (Swiggy / Zomato)</b> Users, restaurants, delivery agents, order tracking, ratings system         |

■ **FOCUS AREAS FOR EACH PROBLEM:** Identify entities (classes) → Design relationships → Handle edge cases → Write clean methods → Test all scenarios

## ■ LLD INTERVIEW APPROACH — 6-Step Framework

| Framework | 6-STEP APPROACH TO SOLVE ANY LLD PROBLEM                                                               |
|-----------|--------------------------------------------------------------------------------------------------------|
| 1         | <b>Understand Requirements</b> Ask questions, clarify scope, identify constraints and assumptions      |
| 2         | <b>Identify Key Entities</b> Find classes needed — nouns in the problem statement = your entities      |
| 3         | <b>Define Class Structure</b> Attributes, methods, constructors for each class — think data + behavior |
| 4         | <b>Establish Relationships</b> Inheritance, composition, associations between classes — draw it first  |
| 5         | <b>Write Clean Code</b> Apply SOLID principles, use design patterns where appropriate                  |
| 6         | <b>Handle Edge Cases</b> Error handling, input validation, boundary conditions, concurrency            |

## ■■ TOOLS & TECHNOLOGIES

|                 |                                      |                                            |
|-----------------|--------------------------------------|--------------------------------------------|
| Languages       | Java, C++, Python, JavaScript        | Use any language you're comfortable with   |
| Diagrams        | Draw.io, Lucidchart, Visual Paradigm | Sketch class diagrams before writing code  |
| IDEs            | IntelliJ IDEA, VS Code               | Write clean, well-organized, readable code |
| Version Control | Git / GitHub                         | Commit regularly — show your work history  |

## ■ RESOURCES & LEARNING

|                    |                                                                                                                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ■ FULL LLD COURSE  | <b>Syntax Error — LLD Course</b> by @code.abhii07<br><a href="https://syntaxerrorrr.com">syntaxerrorrr.com</a><br>Complete LLD notes, design patterns explained with examples, solution walkthroughs for all 5 real problems — everything in one place.                                                                                      |
| ■■ VIDEO SOLUTIONS | <b>LLD YouTube Playlist</b> by Syntax Error<br><a href="https://youtube.com/@code.abhii07">youtube.com/@code.abhii07</a><br>Full LLD video course — All 5 phases + 5 complete problem solutions with code. Best for visual learners who want to see design in action.                                                                        |
| ■ PATTERNS BIBLE   | <b>Design Patterns — Refactoring Guru</b> by <a href="https://refactoring.guru">refactoring.guru</a><br><a href="https://refactoring.guru/design-patterns">refactoring.guru/design-patterns</a><br>Visual design patterns guide with examples and real-world scenarios. The best resource to deeply understand all 23 Gang of Four patterns. |

## ■ PHASE MILESTONE TRACKER

| Phase   | Key Deliverables                                            |
|---------|-------------------------------------------------------------|
| Phase 1 | Student class, Bank Account — full OOP implementation       |
| Phase 2 | Refactor Phase 1 code applying all 5 SOLID principles       |
| Phase 3 | Class diagrams for 3 real systems — draw before coding      |
| Phase 4 | Implement 5 design patterns with real-world use cases       |
| Phase 5 | 5 complete LLD problems — Parking, Library, Elevator + more |

|                     |                                                                        |                                                                                     |
|---------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>SYNTAX ERROR</b> | Our Website<br><a href="https://syntaxerrorr.com">syntaxerrorr.com</a> | Our Instagram<br><a href="https://www.instagram.com/code.abhii07">@code.abhii07</a> |
|---------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------|

*"Clean code isn't about writing clever code — it's about writing code your team understands. Design for humans first, machines second. ■■"*